

3rd May: React Refresher Continuation Day-2

- Jaani

What is virtual dom in react? 1

Virtual dom is a light weight representation in memory.
React uses this virtual dom to update actual dom.

How it works.

1. Initial render (React creates virtual DOM, and uses it to make changes in actual DOM.)
2. State/Props update (a new virtual dom is made)
3. Diffing: Tell me the differences b/w old and new DOM
4. Minimal changes: Find changes that are to be made
5. Update real dom: Update real dom acc to changes.

Assignment: Write blog on virtual DOM under the hood.

Hooks

Hooks are functions that helps components use state, lifecycle features and other React capabilities. eg,

useState
useEffect
useMemo
useCallback

useRef

useContext.

You can also make your own custom hooks.

Rules for Hooks

2

1. Hooks are defined and called on top level
2. Hooks can't be called inside `fun`

`useState` (when you want to manage state inside component)

⇒ State is data that belongs to a component and changes over time, causing UI to update. So, when state changes → react re-renders component → UI updates automatically.

Code:

```
import { useState } from 'react';

function App() {
  const [state, setState] = useState(initial value)

  return (
    <div>
    </div>
  );
}

export default App;
```

⇒ To make an increment at button click

```
function App() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <h1> {count} </h1>
      <button onClick = {() => setCount(count + 1)} > + </button>
    </div>
  );
}

export default App;
```


⇒ Always use `setState` to update instead of manually updating.

Light / Dark Mode

When toggle button is clicked.

If Light → toggle to dark } do css changes i.e. to state
If dark → toggle to light }

Code :

```
import React, { useState } from "react";
```

```
const App = () => {
```

```
  const [darkMode, setDarkMode] = useState(false);
```

```
  fun onToggleTheme() {
```

```
    setDarkMode(!darkMode)
```

```
    console.log(darkMode)
```

```
  }
```

```
  return (
```

```
    <div
```

```
      style = { {
```

```
        height: "100vh",
```

```
        <h1>Mode { darkMode ? "Switch to light" : "Switch to Dark" }
```

```
        <button onClick = { onToggleTheme } > Toggle Theme
```

```
      </button>
```

```
    </div>
```

```
  )
```

```
};
```

```
export default App;
```

UseEffect

It is a hook used to run code after a component renders. UseEffect lets us perform side effects in our components. UseEffect takes a callback and a dependency array.

Three types of useEffect

1. Without dependency array

2. With dependency array

3. Empty dependency array

1. Without dependency array (runs on first render and on every re-render)

```
useEffect(() => {
```

```
  log("Hello, I am rendered");
```

```
});
```

2. With empty dependency array (will be triggered only once on mount and not on change in state). Changes when you reload.

```
useEffect(() => {
```

```
  console.log("runs once");
```

```
}, []);
```

3. With dependency array (runs on first render and whenever count changes)

```
useEffect(() => {
```

```
  log("Hello from useEffect");
```

```
}, [count, setCount])
```


To fetch data from Github

5

```
import React, { useState, useEffect } from "react";
```

```
const App = () => {
```

```
  const [isLoading, setIsLoading] = useState(false);
```

```
  const [data, setData] = useState(null);
```

```
  useEffect(() => {
```

```
    const fetchGithubData = async () => {
```

```
      try {
```

```
        setIsLoading(true);
```

```
        const response = await fetch("https://api.github.com
```

```
        const data = await response.json();
```

```
        setData(data)
```

```
      } catch (error) {
```

```
        console.log(error);
```

```
      } finally {
```

```
        setIsLoading(false)
```

```
      }  
      fetchGithubData();
```

```
    }, [])
```

```
    return <GithubProfile data={data} isLoading={isLoading} />;
```

```
  };
```

Add CSS accordingly

```
export default App;
```

Event Handling

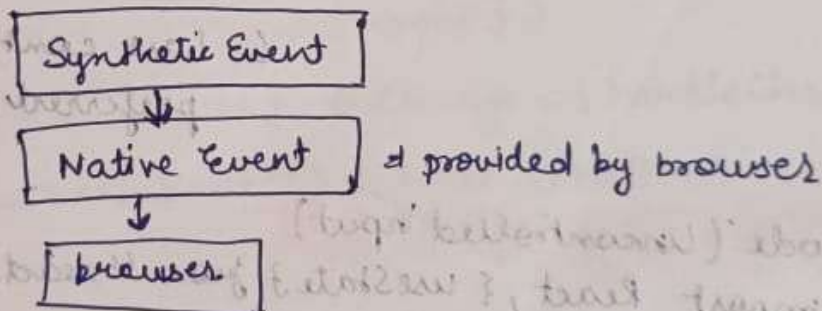
It is how we respond to user interactions.

Type of Events

1. Mouse Events
2. Keyboard Events
3. Form events
4. Touch events
5. UI events (on Scroll, re-size)

In React, it is called synthetic events. A Synthetic event is React's wrapper around the browser's native event.

This is done so things work same across all browsers.



Code,

```
import React from "react"
const App = () => {
  const onclick = (event) => {
    console.log(event) // prints Synthetic Base Event
  }
}
```

Here "event" is Not a raw DOM event, It is a Synthetic event

Form Handling

Preferred approach

Form Handling

controlled

uncontrolled

1. Input is controlled by react's state
2. React is single source of truth
3. easy validation
4. works well with react's ecosystem

1. Input manages its own state meaning, DOM handles the input, React just reads it
2. No react state
3. Value accessed only when needed
4. Hard to validate
5. Not reactive
6. Less controlled, not preferred.

Code (Uncontrolled input)

```
import React, { useState } from "react";
```

```
const App = () => {
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault()
```

```
    log(e.target.elements.name.value)
```

```
    return ( <form onSubmit = {handleSubmit}>
```

```
      <input name = "name" />
```

```
      <button type = "submit"> Submit </button>
```

```
    </form> )
```

```
  )
```

```
  export default App;
```

66 Code : For controlled input

18

```
import { useState } from "react",

function App () {
  const [name, setName] = useState("");

  function handleChange (e) {
    setName (e.target.value);
  }

  function handleSubmit (e) {
    e.preventDefault();
    log(name);
  }

  return (
    <form onSubmit = {handleSubmit}>
      <input value = {name} onChange = {handleChange}>
      <button type = "submit"> Submit </button>
    </form>
  );
}
```

Assignment : Make a form to input email and name.

Follow: jaaaani404

Happy Learning!